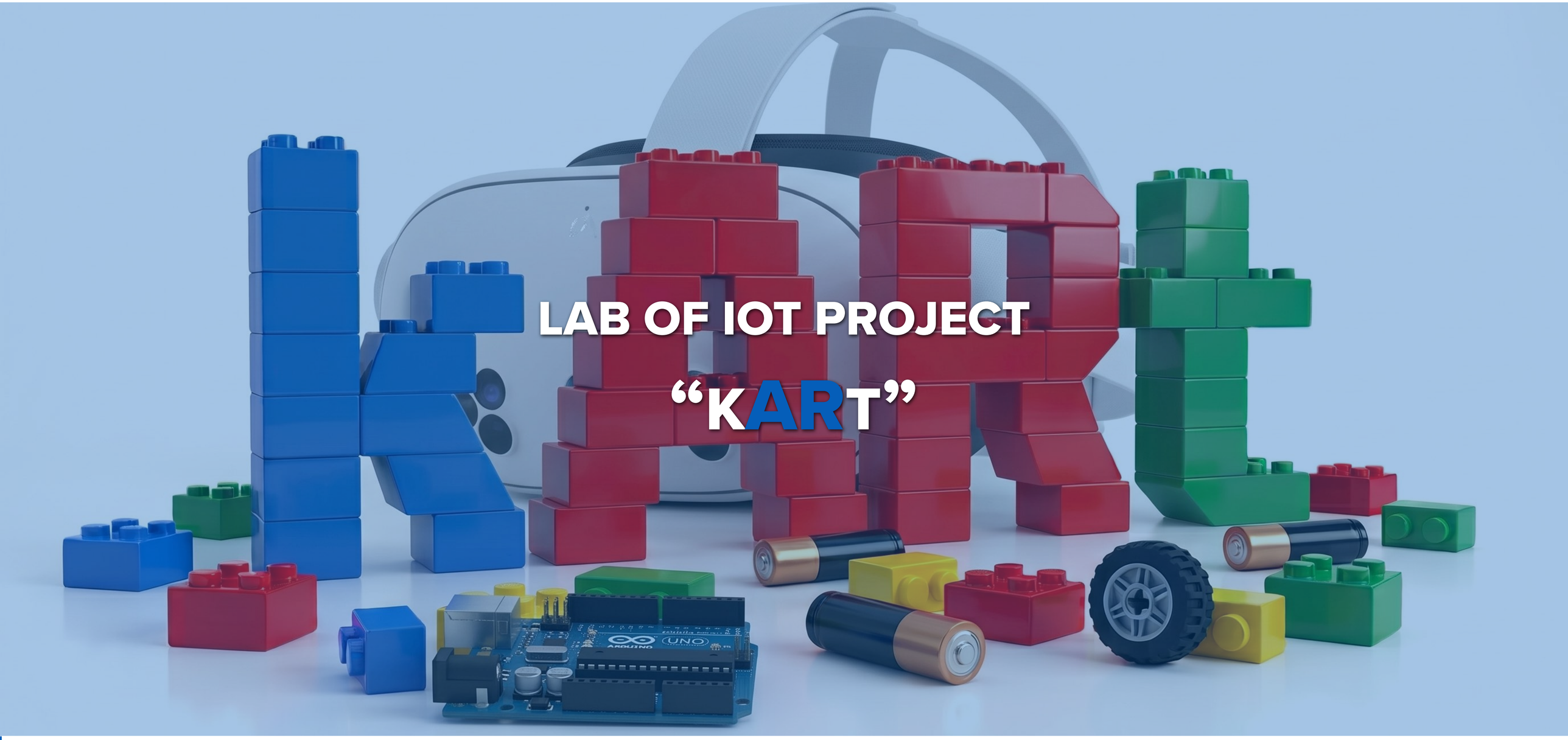


LAB OF IOT PROJECT

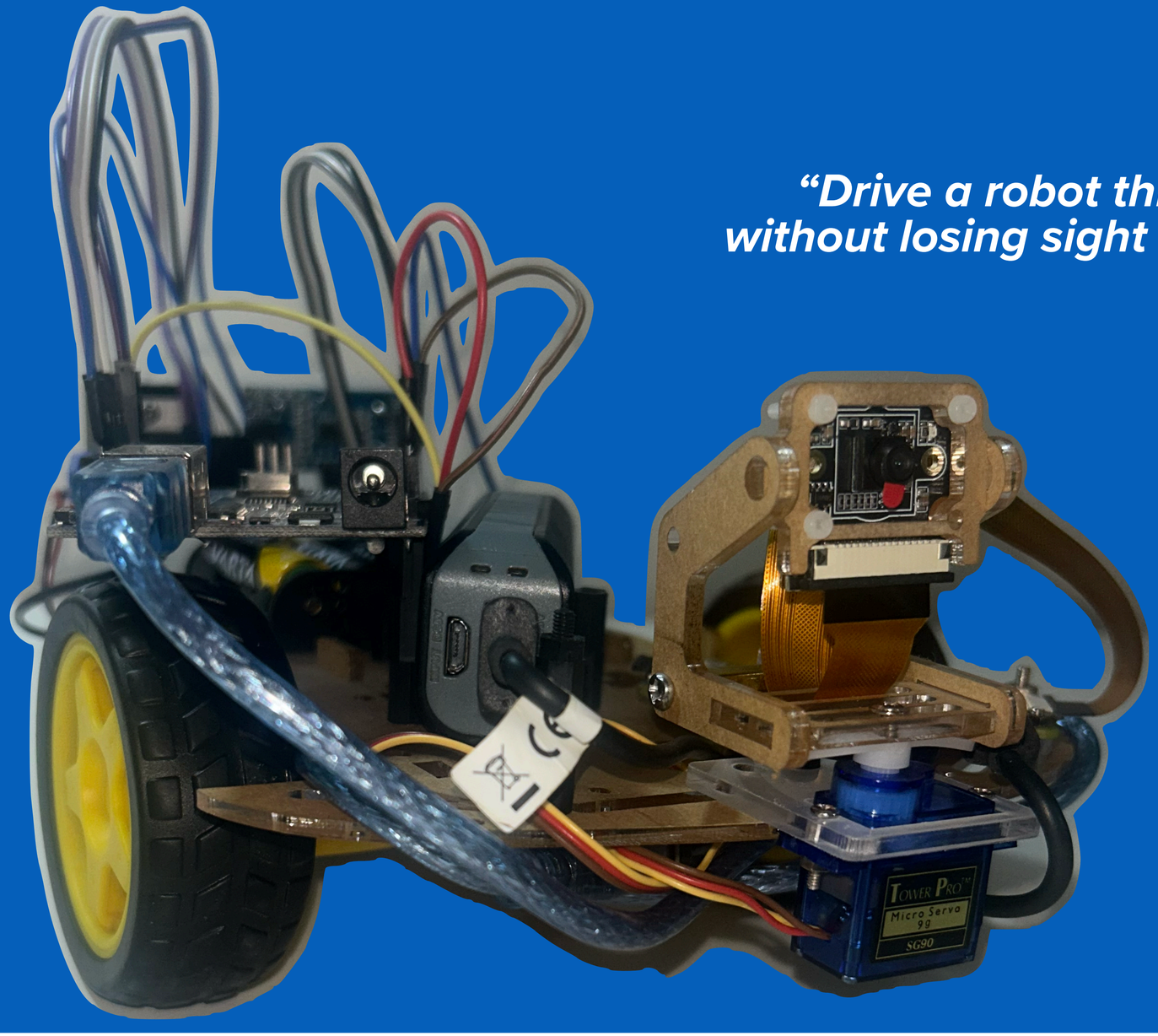
“KART”

PROFESSOR: MASSIMO FICCO

A.Y. 2026/2027

CANDIDATE: ENRICO MADONNA

THE IDEA



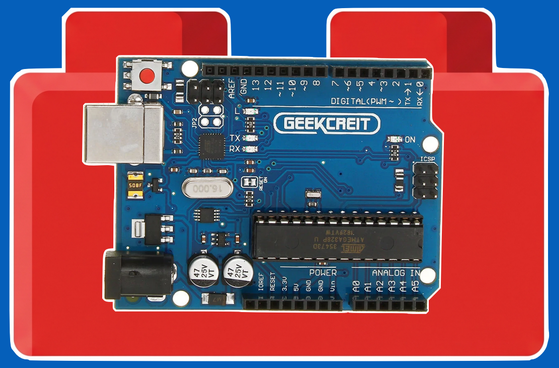
“Drive a robot through its eyes,
without losing sight of the real world.”

POTENTIAL APPLICATION

First approach to Driving

Kids (10+) learn driving concepts by playing: steering, spatial awareness, parking with the rear sensor. All of that at home with Parents supervision

SENSORS & ACTUATION NODE



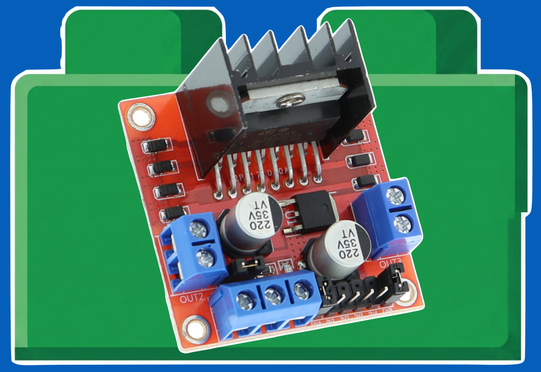
Geekcreit Arduino UNO R3 Compatible

Real-time controller of the kart. Receives driving commands over serial and drives the motors and the servo.

HC-SR04

Parking sensor mounted on the back. Measures the distance to obstacles every 50 ms (median of 3 samples). Below 15 cm, reversing is blocked automatically.

SENSORS & ACTUATION NODE

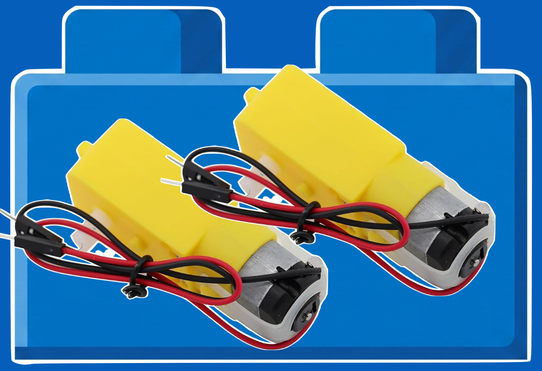


Driver L298N

Dual H-bridge driving the two DC motors. Receives the signals from the Arduino and sets direction and speed of each motor, powering them from the 4×AA pack.

Rotates the onboard camera by ±45° (left / right), giving the pilot a wider view of the surroundings.

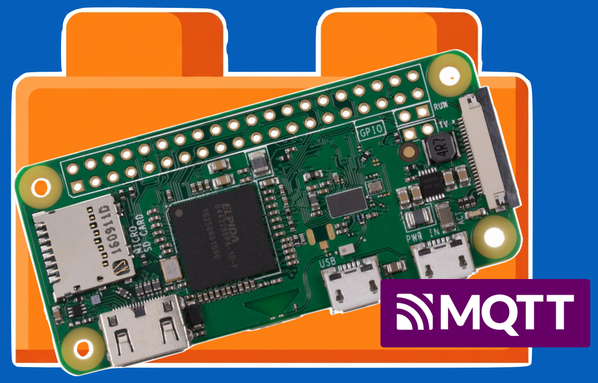
Servo SG90



TT gear motors

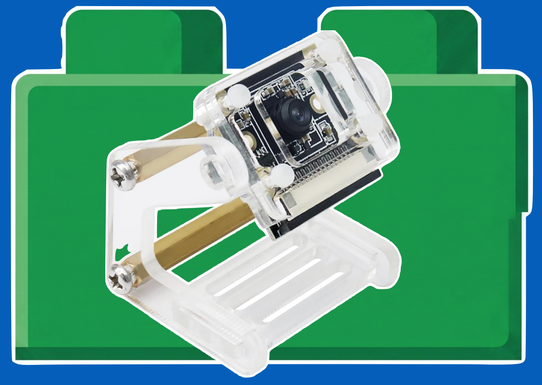
DC motors with 1:48 gearbox, one per rear wheel. Differential drive: the kart steers by varying the speed of the two wheels, with no mechanical steering.

GATEWAY NODE



Raspberry Pi Zero W

Gateway between the kart and the network. Forwards UDP commands to the Arduino over serial, streams the camera video and publishes the telemetry.



Camera IMX219 120°

First-person camera mounted on the servo. The video is streamed to the headset.

APPLICATION LAYER

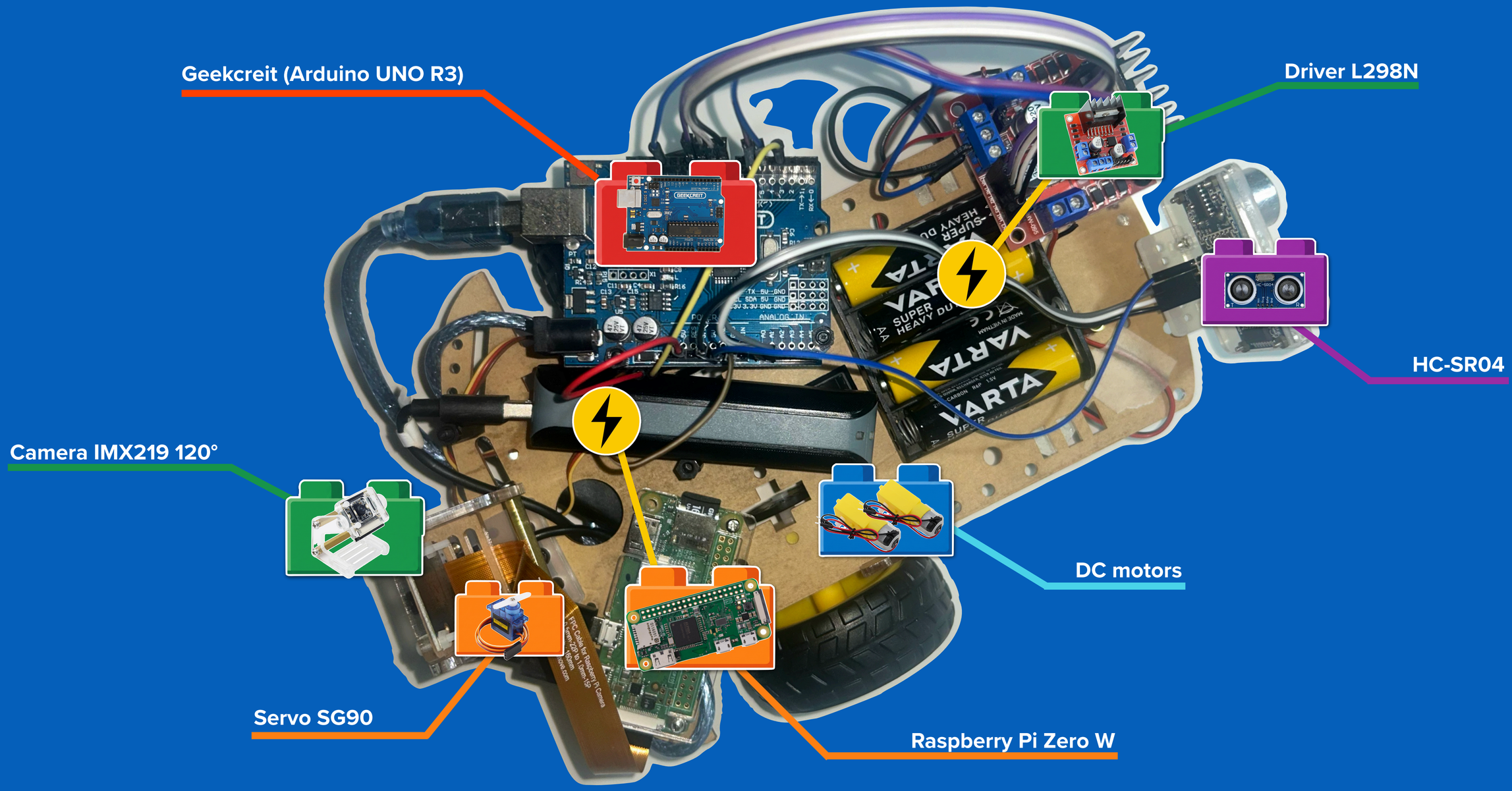
META Quest 3S

Mixed Reality cockpit. Passthrough keeps the real room visible, with the FPV video, the telemetry HUD and the parking sensor shown on top. The kart is driven with the controllers.

PC/Mac

Monitoring dashboard, subscribed to the Pi’s MQTT broker. Shows real-time telemetry, node status and system metrics. It is optional: the kart works without it.

KART OVERVIEW



FIRMWARE - ARCHITECTURE

Let’s take a look at the kart firmware installed on the Arduino.

On every loop:

wdt_reset(): tells the hardware watchdog that the firmware is alive

leggiSeriale(): immediately reads the incoming commands from the Pi

Every 50 ms (20 Hz): measures the distance, updates the drive logic and sends telemetry. The drive logic is re-checked even when no commands arrive, so timeouts and obstacles are always detected.

aggiornaServo(): moves the camera gradually towards the requested angle

```
void loop() {
  wdt_reset();          // hardware watchdog: "I'm alive"
  leggiSeriale();       // read commands from the Pi

  unsigned long ora = millis();
  if (ora - ultimoSensoreMs >= PERIODO_SENSORE_MS) {   // every 50 ms (20 Hz)
    ultimoSensoreMs = ora;
    aggiornaDistanza();
    applicaComando();     // re-check timeout and distance even without new commands
    inviaTelemetria();
  }
  aggiornaServo(ora);   // smooth camera pan (ramp)
}
```

FIRMWARE - SAFETY LOGIC

Safety on three levels

Level	Fault covered	Reaction
Command watchdog (300 ms)	Wi-Fi or Pi lost	Motors stopped, camera centered
Lock after FAILSAFE	Sudden restart	Restarts only with throttle = 0
Reverse braking	Obstacle behind	< 30 cm slows down, < 15 cm STOP
AVR hardware watchdog (500 ms)	Firmware hang	Reset with motors stopped

```
// 1. Command watchdog
if (!haComando || ora - ultimoComandoMs > TIMEOUT_CMD_MS) {
  bloccato = true;
  stato = FAILSAFE;
  panTarget = 0;
  motoriStop();
  return;
}
// after FAILSAFE, restart only with throttle = 0 (no sudden starts)
if (bloccato) {
  if (cmdThrottle != 0) {
    stato = FAILSAFE;
    motoriStop();
    return;
  }
  bloccato = false;
}

// 2. Reverse braking
long thr = cmdThrottle;
// hysteresis: if already in STOP, the exit threshold is slightly farther
int sogliaStop = (stato == STOP) ? stopCm + ISTERESI_CM : stopCm;
stato = OK;
if (thr < 0 && distCm >= 0) {
  if (distCm < sogliaStop) {
    thr = 0;
    stato = STOP;
  } else if (distCm < WARN_CM) {
    // reverse speed from 30% (close) to 100% (at 30 cm)
    long perc = 30 + 70L * (distCm - stopCm) / (WARN_CM - stopCm);
    thr = thr * perc / 100;
    stato = WARN;
  }
}
```

FIRMWARE - SENSOR PROCESSING & DRIVE

```
// Median of the last 3 readings: discards isolated wrong measurements.
// "No echo" (-1) counts as infinite distance.
void aggiornaDistanza() {
  letture[idxLettura] = misuraCm();
  idxLettura = (idxLettura + 1) % 3;

  int v[3];
  for (uint8_t i = 0; i < 3; i++) v[i] = letture[i] < 0 ? 32767 : letture[i];
  int mediana = max(min(v[0], v[1]), min(max(v[0], v[1]), v[2]));
  distCm = mediana == 32767 ? -1 : mediana;
}
```

Median filter on the HC-SR04 distance

Every 50 ms the rear sensor measures the distance to the obstacle.
The value used is the median of the last 3 readings: a single wrong value, common with ultrasound (lost echo or bounce), is discarded without slowing down the response.

If no echo comes back, the distance counts as infinite, i.e. no obstacle.

```
  // Differential mixing
  long sx = thr + cmdSteer;       // left wheel
  long dx = thr - cmdSteer;       // right wheel
  long m = max(abs(sx), abs(dx));
  if (m > 255) {                  // normalize, keeping the ratio
    sx = sx * 255 / m;
    dx = dx * 255 / m;
  }
  sx = sx * maxSpeed / 255;       // apply remote speed limit
  dx = dx * maxSpeed / 255;
```

Differential drive

The kart has no mechanical steering: to turn, one wheel spins faster than the other. The firmware combines the controller’s throttle and steering into a speed for each wheel. If one of them would exceed the maximum, both are scaled down proportionally, so the kart keeps the same curve.

Remote speed limit: max_speed, set via MQTT, scales the final output.

COMMUNICATION

The path of a command

Quest — UDP 30 Hz —▶ Pi — Serial —▶ Arduino

Quest ◀— UDP 20 Hz — Pi ◀— Serial — Arduino

Each command has a sequence number: old packets are dropped and the Quest measures the round-trip time.

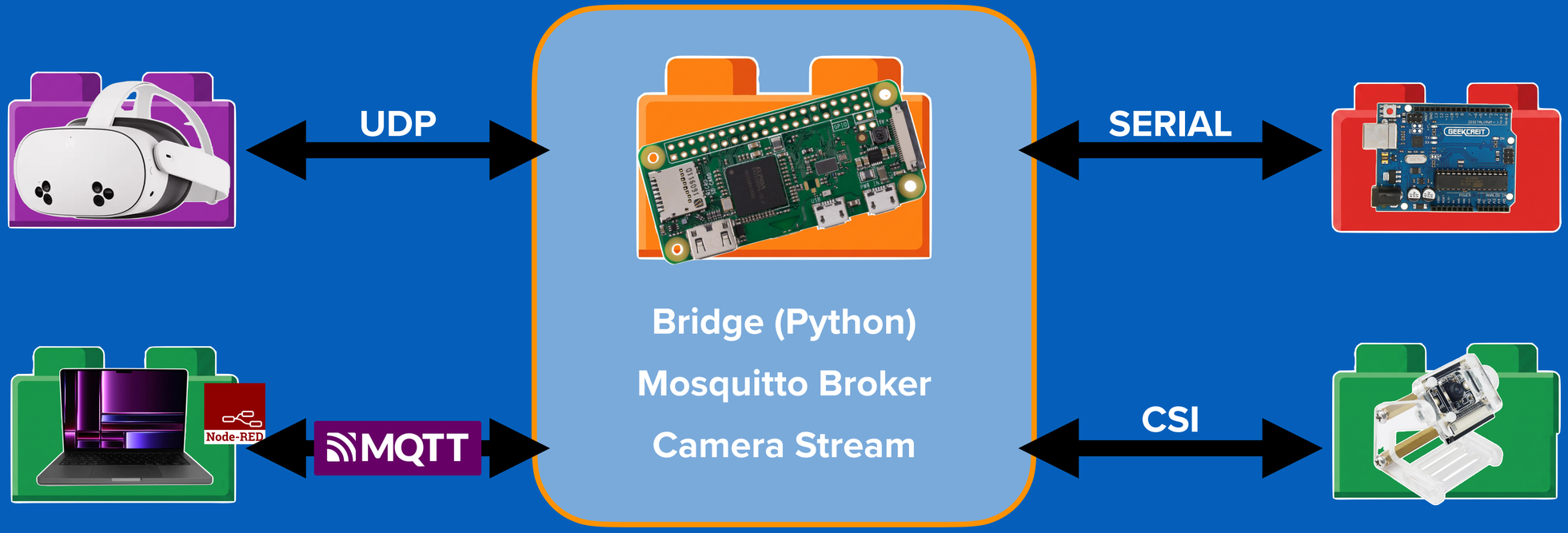
	UDP: driving	MQTT: monitoring
Why	Minimal latency, no handshake	Publish/subscribe, decoupled nodes
Flow	Quest ⇄ Pi	Pi, Quest and Node-RED ⇄ broker on the Pi
If it fails	The watchdog stops the kart	Driving goes on: only monitoring is lost

Main MQTT topics

1. arkart/kart/telemetry: distance, state, commands, RSSI, CPU temperature (2 Hz)
2. arkart/kart/events: COLLISION_PREVENTED, FAILSAFE
3. arkart/kart/status, arkart/quest/status: online / offline via Last Will
4. arkart/quest/metrics: RTT, video FPS, lost commands, headset battery
5. arkart/config: max speed and braking threshold set remotely

GATEWAY - RASPBERRY PI

The network brain of the kart. Translates the Quest commands for the Arduino, hosts the MQTT broker and streams the camera video.



UNITY APP

MR app built with Unity on OpenXR. Passthrough can keep the real room visible, while a HUD with the kart telemetry appears in front of the pilot.

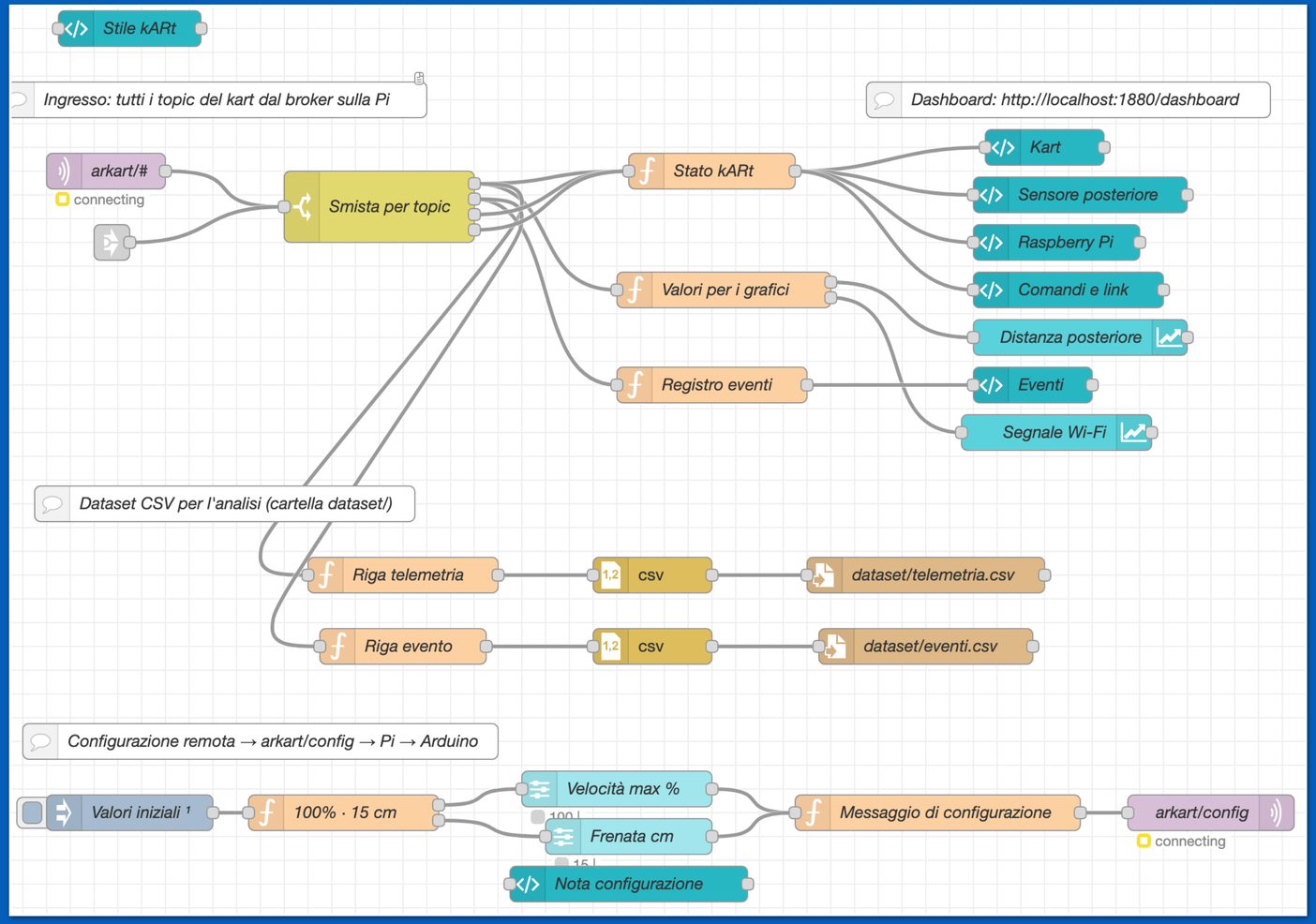
Controls

Controller	Action
Right / left trigger	Forward / reverse
Left stick	Steering
Right stick	Camera pan (click = center)
B button	Emergency stop
X button	Passthrough toggle

HUD

1. Connection and kart status (OK / WARN / STOP / FAILSAFE)
2. Throttle, steering and camera angle bars
3. Parking sensor: segmented bar green → yellow → red
4. RTT and packets sent / received

NODE-RED



Monitoring

Node-RED subscribes to arkart/# on the Pi's broker and routes each topic to a dashboard: kart state, rear sensor, Pi health, live charts and an event log. Two sliders publish max speed and braking threshold on arkart/config.

Data logging

Node-RED is the only node that stores data: every telemetry message and every event is appended to telemetria.csv and eventi.csv, ready for offline analysis.

NODE-RED DASHBOARD

Stato kart

Pronto

comandi ricevuti, nessun ostacolo dietro

Kart	online
Quest	online
Ultimo dato	adesso

Sensore posteriore

Distanza dall'ostacolo

libero

rallenta sotto 30 cm · frena sotto 15 cm

Link Quest

RTT comandi	46 ms
Video	8 fps
Comandi persi	0
Batteria visore	28%

Raspberry Pi

Temperatura CPU	47.6 °C
Segnale Wi-Fi	-41 dBm ottimo
Acceso da	10 min 7 s

Distanza posteriore nel tempo

Eventi

1866 frenate · 1882 fail-safe — attiva suono

23:56:35	Fail-safe · comandi interrotti, motori fermi
23:52:56	Fail-safe · comandi interrotti, motori fermi
23:51:49	Fail-safe · comandi interrotti, motori fermi
23:51:45	Fail-safe · comandi interrotti, motori fermi
23:51:43	Fail-safe · comandi interrotti, motori fermi
23:49:37	Fail-safe · comandi interrotti, motori fermi
22:55:18	Fail-safe · comandi interrotti, motori fermi

Segnale Wi-Fi della Pi

Comandi e configurazione

Acceleratore 91%

Sterzo centro

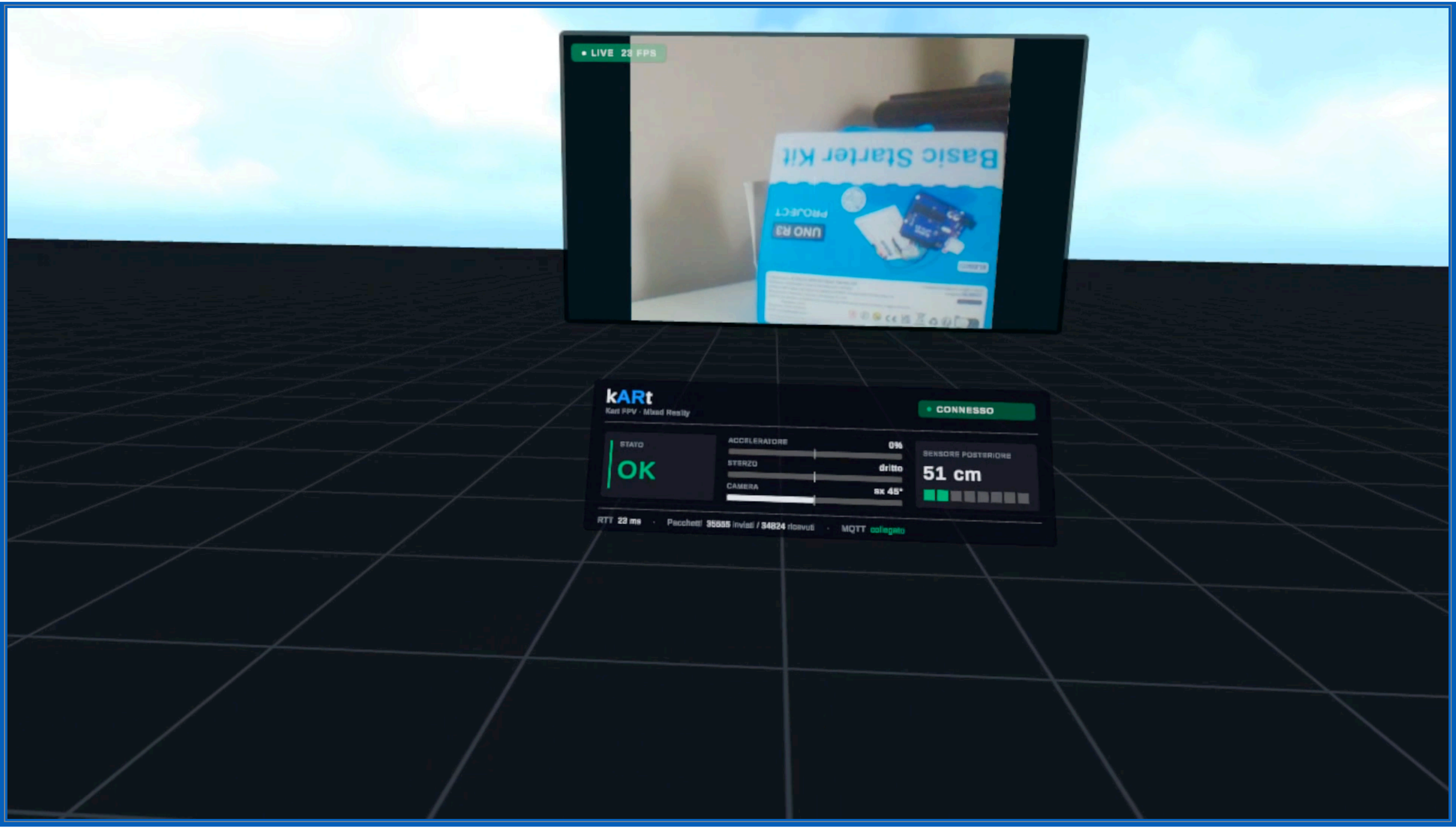
Camera dx 20°

Velocità max %

Frenata cm

Pubblicati su `arkart/config` : l'Arduino li applica subito.

DEMO


LIVE 23 FPS
Basic Starter Kit
UNO R3
PROJECT
kARt
Kart FPV · Mixed Reality
CONNESSO
STATO
OK
ACCELERATORE
0%
STERZO
dritto
CAMERA
sx 45°
SENSORE POSTERIORE
51 cm
RTT 23 ms · Pacchetti 35585 inviati / 34824 ricevuti · MQTT collegato

FUTURE WORK

Coordination study

Research on XR for children with coordination difficulties (DCD) is promising*. A field study is possible, measuring driving progress over time (lap times on a track, collisions, parking accuracy) using the telemetry.

Mixed Reality races

Multiple karts in the same room, with finish lines, checkpoints and virtual obstacles overlaid on the real floor. The MQTT architecture fits well: each kart is just one more node on the same broker.

Difficulty levels

Predefined driving profiles (beginner, expert) that automatically tune speed and braking, using the remote configuration.

*Piñar-Lara et al., Disability and Rehabilitation, 2025.

THANKS FOR THE ATTENTION!

GITHUB REPO